RNGD AI Studio

User Manual

RNGD semester proof-of-concept

October 5, 2026

Contents

1 What this is **2**

2 Getting started **2**

3 The interface **2**

4 Mode 1: Multi-Agent Pipeline **3**

4.1 Running it 3

4.2 Watching it run 3

4.3 Reading the results 3

4.4 Sample run (most recent, this app) 3

5 Mode 2: IFC Design Studio **4**

5.1 Using it 4

5.2 Reading the results 4

5.3 Two contrasting designs 5

6 The chat assistant **6**

7 Testing **6**

8 Troubleshooting **7**

9 Where things live **7**

10 Limitations **7**

1 What this is

RNGD AI Studio is one Streamlit app, `src/app.py`, with two modes and a chat assistant that is always visible regardless of which mode you're in.

- **Multi-Agent Pipeline** — ten agents turn an unstructured customer request and raw site conditions into a schematic: a site plan, a floor plate, a conflict report, and one final AI decision.
- **IFC Design Studio** — a faster, parametric generator: move a slider or click a scenario and get an instant 3D building written as a real, standard IFC4 file, scored against ten compliance checks.
- **Assistant** — a chat panel on the right that knows about every component of both modes, answers questions in plain language, and posts its own observations after a run finishes.

All zoning rules and the BaaP module catalog are synthetic sample data. Outputs are planning-level, not code-approved; a licensed professional must review any real design.

2 Getting started

1. `pip install -r requirements.txt`
2. Create `RNGD/.env` with `OPENROUTER_API_KEY` (not committed to git).
3. `python src/make_sample_data.py` (once, generates the sample site sketch).
4. `streamlit run src/app.py` — opens in your browser at `http://localhost:8501`.

3 The interface

The page has three regions, all visible at once on a normal-width window:

Region	Contents
Header	Title banner with live status pills: how many pipeline agents, how many IFC scenarios, the knowledge-base backend and chunk count, and the last decision once a pipeline run has completed.
Sidebar (left)	A Mode switch (Multi-Agent Pipeline / IFC Design Studio) and that mode's controls — everything below the switch changes depending on which mode is selected.
Main column (center)	Whichever mode's tabs and content are showing.
Chat panel (right)	Always the assistant, in every mode, with its own scrollable message history and an input box pinned to the bottom of the page.

4 Mode 1: Multi-Agent Pipeline

4.1 Running it

In the sidebar: pick the sample project (currently one, `project_001`), set the maximum number of simulation iterations (default 3), and click **Run full pipeline**.

4.2 Watching it run

A live status panel opens in the main column and updates in place as each agent finishes — you see the agent name, what it did, how long it took, and a one-line summary, in order, as it happens, not just a table at the end. When the run finishes the panel collapses to a single line showing the final decision.

4.3 Reading the results

Eight tabs appear once a run has completed:

Tab	What's there
Overview	Decision, confidence, conflict count, the narrative, and the two headline images (site plan, typical floor).
Agent trace	Every step as a table: model used, status, latency, summary.
Program & site	The parsed customer program and the computed zoning/site constraints, plus the input sketch.
BaaP & conflicts	The module-matching plan and the full conflict report with severities.
Site simulation	Each physics-simulation iteration's stats and rendered site plan, the critic's suggestions, and a convergence chart if more than one iteration ran.
Floor plates	Ground floor and typical upper floor, side by side.
Final decision	The Arbiter's full narrative, unresolved issues and recommended next steps.
Ask an agent	Pick one agent and ask it a one-off question grounded in its own domain (separate from the chat panel — this talks to a single named agent).

4.4 Sample run (most recent, this app)

Decision	**REVISE** (confidence 95%)
Trace steps	11
Conflicts found	2
Geometry passed (CV QA)	True
Parking	15 / 150 stalls
Lot coverage	46.3%

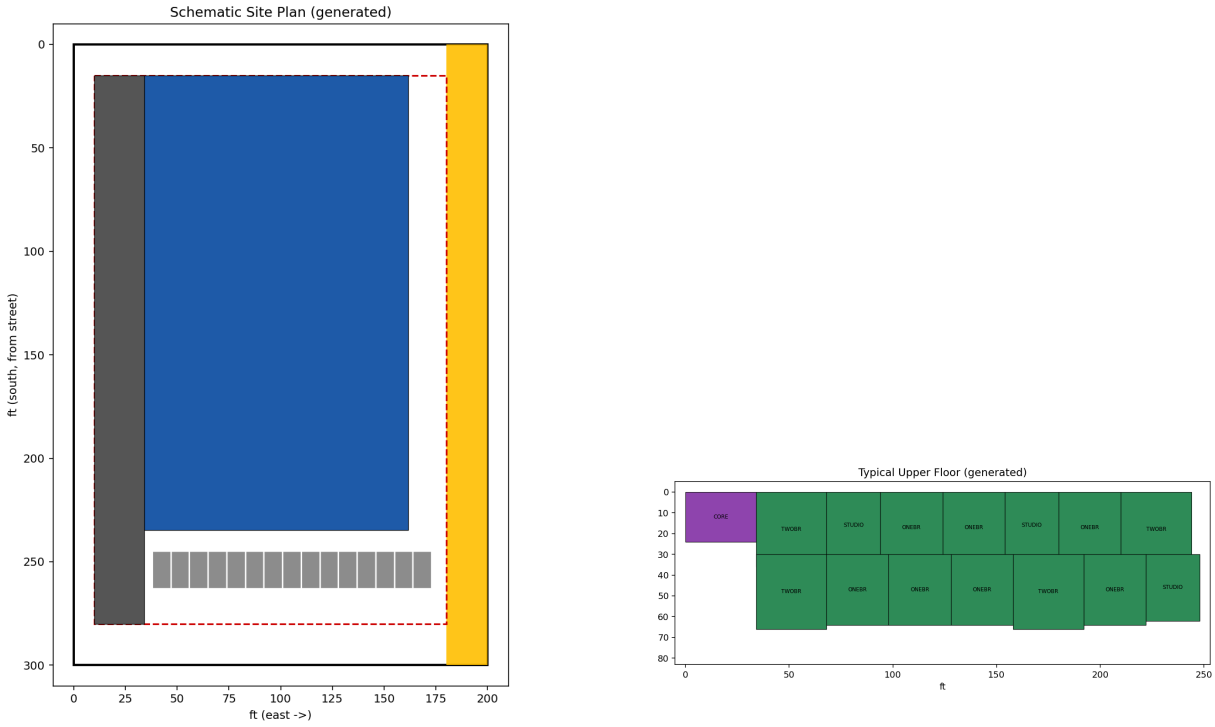


Figure 1: Generated site plan (left) and typical upper floor (right) from the run above.

5 Mode 2: IFC Design Studio

5.1 Using it

Switch the sidebar Mode to **IFC Design Studio**. Either click a scenario button to load a preset in one click, or move the controls yourself: units, residential floors, studio/1BR/2BR mix, zoning preset, parking type (surface or podium), and number of wings (0 = let the program choose). Every change regenerates the design — there is no separate ”run” button, results update as you move a slider.

You can also paste a free-text customer request into the “Paste a customer request” expander and click **Extract requirements** to have the same AI intake agent used by the pipeline fill in units/floors/mix.

5.2 Reading the results

The top strip always shows: units placed, stories, FAR, lot coverage, parking provided/required, and how many of the ten checks failed. Five tabs follow:

Tab	What's there
3D model	An interactive Plotly view of the actual IFC file (drag to rotate, scroll to zoom, click a legend entry to hide a unit type, limit visible floors).
Site plan	A top-down 2D view: lot, easement, buildable envelope, driveway, parking, wings.
Compliance	The ten PASS/FAIL checks as a table, plus a button to have the assistant write a short review of this specific design.
How this was built	A plain-language, step-by-step account of *this* design: the envelope math, the unit mix, why this many wings were chosen, the parking calculation, and the result.
Download	The `.ifc` file and a standalone `.html` 3D viewer you can open in Chrome without the app running.

5.3 Two contrasting designs

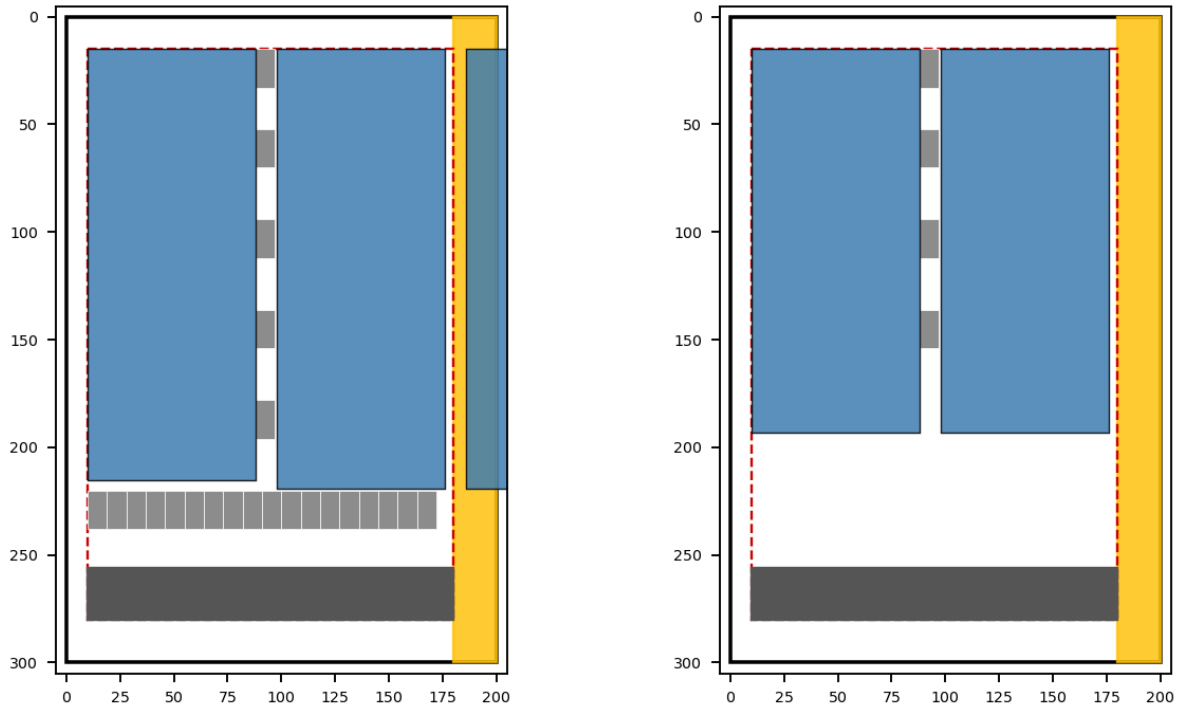


Figure 2: Baseline 120 units, surface parking (left, fails coverage/FAR/parking) vs. the right-sized 60-unit podium design (right, passes all ten checks).

For the full method behind these numbers (formulas, packing algorithm, every scenario), see `reports/ifc_design/ifc_design_report.pdf`.

6 The chat assistant

The panel on the right is available in both modes and keeps its conversation across mode switches. It is told, in its own system prompt, about every agent in the pipeline and every part of the IFC studio, so it can answer general questions ("what is FAR", "what does the Design Critic Agent do") even before you run anything.

Once you do run something, the assistant is given a compact summary of the current state — the parsed program, the buildable envelope, the conflict list, the latest CV metrics and the final decision for the pipeline; the requested parameters, metrics and failed checks for the IFC studio — so its answers are grounded in *your* actual numbers, not generic ones. It also volunteers a short note on its own:

- automatically, right after a pipeline run finishes;
- on demand, from the "Ask the assistant to review this design" button in the IFC studio's Compliance tab (this is on-demand rather than automatic there, since every slider move regenerates a design and an automatic note on every tiny change would be noisy).

Example questions: *"Why did parking fail?"*, *"What's the difference between the two modes?"*, *"If I switch to podium parking, what should I expect?"*, *"What is a double-loaded corridor?"*

7 Testing

`python -m pytest tests/` runs the full suite:

- pipeline agent and IFC-engine logic tests (module counts, compliance scoring, IFC round-trip through IfcOpenShell);
- app-level tests using Streamlit's `AppTest`, which drive the real app: load without error, switch mode and click a scenario button, submit a chat message, and — the slowest one — click *Run full pipeline* in the actual sidebar and check the live-status wiring, the final decision and the proactive chat note all worked.

This last test is what would have caught an earlier bug where the app crashed on a relative launch path; it stays in the suite for exactly that reason.

8 Troubleshooting

Symptom	Fix
`IndexError` on startup mentioning `parents[...]`	Launch from the repo root as `streamlit run src/app.py`, or make sure any `Path(__file__)` use is `.resolve()`d first — already fixed in this app, kept here as a reminder if you copy the pattern elsewhere.
"Port already in use"	Add `--server.port 8502` (or any free port) to the `streamlit run` command.
Chat button greyed out / no reply	Check `OPENROUTER_API_KEY` is set in `.env` and the process was restarted after adding it.
"Extract requirements" fails	The intake agent needs `OPENROUTER_API_KEY`; everything else in the IFC studio works without any key.
First load feels slow	The knowledge-base embedding model downloads/loads once per process start (a few seconds); subsequent reruns reuse it.

9 Where things live

Path	Contents
`src/app.py`	The unified app (this manual).
`src/rngd/agents/`, `orchestrator.py`	The nine pipeline agents + the harness that runs them.
`src/rngd/ifc/`	The IFC studio engine: `design.py` (layout + rules), `ifc_writer.py` (IFC I/O), `viewer3d.py` (3D figure).
`src/rngd/chat.py`	The chat assistant.
`data/`	Sample knowledge base, BaaP catalog, and the one sample project.
`outputs/`	Generated images/files from the last run of each mode.
`reports/`	This manual, the IFC studio's own deep-dive report, and the RVT-reading report.
`tests/`	The full pytest suite described above.
`Testt/`	Separate side experiments that read raw `.rvt` files and a pyRevit wrapper; see `Testt/README.md`.

10 Limitations

- All zoning rules and the BaaP catalog are synthetic; swapping in RNGD's real data is a data change, not a code change.
- The pipeline's simulation/CV loop and the IFC studio's layout algorithm are both planning-level heuristics, not licensed engineering.
- The chat assistant answers from the current state and the knowledge base; it says so when it doesn't have enough grounding rather than guessing.